```
ft_name = "product_3minute_max_price_ft"

if not spark.catalog.tableExists(ft_name) or spark.table(tableName=ft_name).isEmpty():
  raise Exception("problem")
else:
  min_time = sql(f"SELECT MIN(LookupTimestamp) FROM {ft_name}").collect()[0][0]
  max_time = sql(f"SELECT MAX(LookupTimestamp) FROM {ft_name}").collect()[0][0]
  raw_transactions_df = sql(f"""
    SELECT Amount,CustomerID,actual_label as Label,Product,TransactionTimestamp FROM {table_name}
    WHERE TransactionTimestamp >= '{min_time}' AND TransactionTimestamp <= '{max_time}' AND actual_label IS NOT NULL
    """)
```

Figure 7.15 – Defining the feature lookups for the inference model

The transactions in *Figure 7.15* come from the inference table, `packaged_transaction_model_predictions`, created in `CH7-08-Batch Inference`. The baseline model does something similar with the `raw_transactions` table. The baseline model also sets the model description, as shown in *Figure 7.16*.

```
import pandas as pd
import mlflow
mlflow.set_registry_uri("databricks-uc")

model_name = "packaged_transaction_model"
full_model_name = f'{catalog}.{database_name}.{model_name}'
model_description = "MLflow custom python function wrapper around a LightGBM model with embedded
pre-processing. The wrapper provides data preprocessing so that the model can be applied to input
dataframe directly without training/serving skew. This model serves to classify transactions as 0/1 for
learning purposes."

model_artifact_path = volume_model_path + "model_name
dbutils.fs.mkdirs(model_artifact_path)
```

Figure 7.16 – Setting the model description

We are now ready to focus on the model. Let's walk through the rest of the process.

1. Set the model registry to UC with the following line of code: `mlflow.set_registry_uri("databricks-uc")`.
2. Use pip to save a `requirements.txt` file.
3. Create a PyFunc Wrapper for Transaction Model `TransactionModelWrapper`.

Most of the code for the `TransactionModelWrapper` should look familiar to those who have created a model using Sklearn. The initialization function, `__init__(self, model, X, y, numeric_columns, cat_columns)`, accepts a model and DataFrames. The data preprocessing for training and inference data is standardized within the wrapper. The `TransactionModelWrapper` consists of four methods: `init`, `predict`, `preprocess_data`, `fit`.